

BTC Price Forecasting using LSTM and GRU with MLOps Deployment

Final Project Report Category: Finance · Time-Series Forecasting · Deep Learning · MLOps

⚠ **Academic project for educational purposes only. NOT financial advice.** Cryptocurrency prices are highly volatile and inherently unpredictable.

1. Abstract

This project forecasts the next-day **Bitcoin (BTC-USD)** closing price from historical OHLCV data enriched with technical indicators. Two deep recurrent models are built and compared end-to-end under an MLOps workflow:

- **Model V1** — a baseline single-layer **LSTM**.
- **Model V2** — an improved, regularized **GRU** (dropout, early stopping, adaptive learning rate).

The full lifecycle is implemented: data pipeline → training with **MLflow** tracking → **FastAPI** serving → **React** dashboard → **Docker** → **GitHub Actions CI** → **Kubernetes (Minikube)**, and a **live public deployment** (Vercel frontend + Hugging Face Spaces backend).

2. Problem Statement & Real-World Significance

Crypto markets are volatile, 24/7, and sentiment-driven. Short-horizon price forecasting is a canonical **sequence-modelling** problem: given a 60-day window of features, predict the next close. Beyond the (limited) predictive value, the project's real goal is to demonstrate a **production-grade ML lifecycle** — reproducible training, experiment tracking, a versioned API, containerization, CI/CD and orchestration — which is directly transferable to any real forecasting or decision-support system.

3. Dataset

Property	Value
Source	Yahoo Finance via <code>yfinance</code>
Ticker	<code>BTC-USD</code>
Range	2015-01-01 → present (~4,190 daily rows)
Raw features	Open, High, Low, Close, Volume
Engineered	MA-14, RSI-14, MACD, MACD_Signal, Daily_Return
Split	70 % train / 15 % val / 15 % test (chronological)
Scaling	<code>MinMaxScaler</code> fit on train only (no leakage)
Windowing	60-day lookback sequences

Model input shape: **(60, 9)**. Target: next-day scaled `close`, inverse- transformed to USD for evaluation.

4. Methodology

4.1 Feature engineering (`src/preprocessing.py`)

- **MA-14**: 14-day moving average.
- **RSI-14**: relative strength index (EMA formulation).
- **MACD** (+ signal): 12/26/9 EMA convergence-divergence.
- **Daily_Return**: percentage change of close.
- Missing rolling-window values dropped; scaler persisted to `artifacts/scaler.joblib`; a recent feature window cached to `artifacts/latest_window.json` (offline-prediction fallback).

4.2 Model V1 — LSTM baseline (`training/train_v1_lstm.py`)

`LSTM(64) → Dense(1)` · Adam(1e-3) · MSE loss · 50 epochs · batch 32. Deliberately simple to establish a reference.

4.3 Model V2 — improved GRU (`training/train_v2_gru.py`)

`GRU(64) → Dropout(0.2) → Dense(32, relu) → Dropout(0.2) → Dense(1)` . Improvements over V1: **GRU gating, dropout regularization, EarlyStopping (restore best weights), ReduceLROnPlateau, dense head**. Up to 120 epochs; early-stopped at epoch 17 (best weights restored).

5. Results

Example run on data through **2026-06-25** (numbers vary slightly as live data grows). The dashboard's **Model Comparison** tab reads these live from `artifacts/metrics_v*.json`.

5.1 Test-set metrics

Metric	V1 — LSTM	V2 — GRU	Winner
MAE (USD)	5,763.94	4,305.68	V2 
RMSE (USD)	6,770.52	5,172.28	V2 
R ²	0.8357	0.9041	V2 
Directional accuracy	0.4952	0.5064	~tie

5.2 Training summary

	V1 LSTM	V2 GRU
Epochs run	50	42 (early-stop @ 17)
Best val-loss (MSE)	3.70e-4	4.85e-4
Train time	~247 s	~167 s
Parameters	fewer layers, no regularization	dropout + dense head

5.3 Interpretation

- **V2 (GRU) wins** on every price-level metric: ~25 % lower MAE/RMSE and R² 0.90 vs 0.84. GRU's 2-gate design is more parameter-efficient and, with dropout + early stopping, generalizes better on this modest daily dataset.
- **Directional accuracy ≈ 0.50 for both** — neither beats a coin-flip on next-day up/down. Strong RMSE on a near-random-walk series mostly reflects "tomorrow ≈ today" and does **not** imply a tradeable edge. This is expected and is stated honestly (see Limitations).

Artifacts: `training_loss_v{1,2}.png`, `prediction_vs_actual_v{1,2}.png`, `metrics_v{1,2}.json`, `history_v{1,2}.json`, `eval_v{1,2}.json`, `correlation.json`, `price_history.json`.

6. MLOps & System Architecture

```
Yahoo Finance → preprocessing → train V1/V2 → MLflow (params/metrics/artifacts)
      |
      |→ models/*.keras + artifacts/*.json
      |
      v
FastAPI (deployment/api.py) → React dashboard (frontend/)
      |
      |→ Docker → GitHub Actions CI → Kubernetes (Minikube)
      |
      |→ Live: Hugging Face Spaces (API) + Vercel (dashboard)
```

6.1 Experiment tracking — MLflow

Experiment `BTC_Price_Forecasting`. Each run logs `model_version`, `lookback_window`, `epochs`, `batch_size`, `learning_rate`, `architecture` (+ `dropout`, `improvements` for V2) and metrics `MAE/RMSE/R2/directional_accuracy` plus training & validation loss; the model file, loss plot, prediction plot and scaler are logged as artifacts. View: `mlflow ui --port 5000`.

6.2 Serving — FastAPI (`deployment/api.py`)

Method	Path	Purpose
GET	<code>/</code>	metadata + endpoint list
GET	<code>/health</code>	model/scaler existence
POST	<code>/predict</code>	next-price + Up/Down trend + signal + model version
GET	<code>/metrics</code>	V1/V2 metrics + run params + verdict
GET	<code>/diagnostics</code>	loss curves + prediction-vs-actual arrays
GET	<code>/correlation</code>	9-feature correlation matrix
GET	<code>/price-history</code>	recent closes for the forecast chart
GET	<code>/runs</code>	MLflow-style run cards

`/predict` accepts 3 input modes: a full 60-row `sequence`, a single record (appended to live history), or `use_latest`. CORS open for the browser frontend.

6.3 Frontend — React dashboard (`frontend/`)

Vite + React + Tailwind + Recharts, 4 tabs: **Forecast**, **Model Comparison** (MAE/RMSE bars, R^2 gauges, correlation heatmap, LSTM-vs-GRU deep-dive), **Training Diagnostics** (loss + pred-vs-actual), **Experiments** (run cards).

6.4 Docker

Dockerfile (python:3.11-slim, honors \$PORT) + docker-compose.yml (api on 8000 + MLflow on 5000).

6.5 CI/CD — GitHub Actions (.github/workflows/ci.yml)

On push/PR to main + manual dispatch: Python 3.11, install requirements-ci.txt , **syntax-check** the 8 core modules, ruff, verify Dockerfile + K8s manifests exist, validate YAML, run structure tests. Intentionally lightweight — **no TensorFlow training in CI**.

6.6 Kubernetes (Minikube)

kubernetes/deployment.yml (btc-price-api , image btc-price-final:v1 , imagePullPolicy: Never , readiness/liveness on /health) + kubernetes/service.yml (NodePort 30080).

7. Live Deployment

Component	URL
Dashboard (Vercel)	https://btc-lstm-gru-forecasting.vercel.app
API (Hugging Face Spaces)	https://obaiz-btc-forecasting-api.hf.space
API docs	https://obaiz-btc-forecasting-api.hf.space/docs
Source (GitHub)	https://github.com/Obaiz1/LSTM-vs-GRU-BTC-Price-Prediction

Frontend builds with VITE_API_BASE pointing at the Space; GitHub is connected to Vercel for auto-deploy. On Spaces, /predict uses a cached feature window because Yahoo Finance is unreachable there (live elsewhere).

8. Requirements Compliance

Requirement	Status	Evidence
Keep Streamlit app + original models	✓	app.py , *_model.keras
Model V1 + V2	✓	models/btc_lstm_v1.keras , btc_gru_v2.keras
MLflow tracking	✓	BTC_Price_Forecasting experiment, mlruns/
FastAPI / , /health , /predict	✓	deployment/api.py (+5 endpoints)
Docker	✓	Dockerfile , docker-compose.yml
GitHub Actions CI/CD	✓	.github/workflows/ci.yml (green)
Kubernetes / Minikube	✓	kubernetes/*.yaml (demoed)
README + screenshot checklist	✓	README.md , docs/commands_for_screenshots.md
Disclaimer (not financial advice)	✓	API, UI footer, docs
Technical indicators (RSI/MACD/MA/returns)	✓	src/preprocessing.py
Tests	✓	tests/ — 6 passing
Live deployment (bonus)	✓	Vercel + Hugging Face

9. How to Reproduce

```
# 1. Environment (Python 3.11 – TensorFlow does not support 3.14)
python -m venv venv && source venv/bin/activate      # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 2. Train both models (writes models/, artifacts/, MLflow runs)
python training/train_v1_lstm.py
python training/train_v2_gru.py

# 3. Track experiments
mlflow ui --host 0.0.0.0 --port 5000                # http://localhost:5000

# 4. Serve the API
uvicorn deployment.api:app --host 0.0.0.0 --port 8000

# 5. Run the dashboard
cd frontend && npm install && npm run dev            # http://localhost:5173

# 6. Docker
docker build -t btc-price-final:v1 . && docker run -p 8000:8000 btc-price-final:v1

# 7. Kubernetes (Minikube)
minikube start
eval $(minikube docker-env) && docker build -t btc-price-final:v1 .
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
```

10. Limitations

1. **No tradeable edge** — directional accuracy ≈ 0.50 ; good RMSE $\approx$ persistence ("tomorrow $\approx$ today") on a near-random-walk series.
2. **Univariate-ish** — price + technical indicators only; no order-book, on-chain, macro or sentiment signals.
3. **Daily granularity, single horizon** — predicts T+1 only.
4. **Regime shifts** — crypto is non-stationary; past patterns may not persist.
5. **"Confidence"** is a coarse trend-strength heuristic, not a probability.
6. **Hosted predictions** use a cached data window (Yahoo Finance blocked on the free Space network).

11. Conclusion

The **improved GRU (V2)** outperformed the **LSTM baseline (V1)** on all price-level metrics (R^2 0.90 vs 0.84) while training faster, thanks to GRU's efficient gating plus regularization and early stopping. More importantly, the project delivers a **complete, reproducible MLOps pipeline** — from data and

experiment tracking through a versioned API, container, CI/CD, Kubernetes, and a live public deployment — which is the transferable, real-world contribution.

Disclaimer: This is an academic project. It is **not financial advice**. Do not use it for real trading decisions.